

Plano de Implantação e MLOps de Alta Escala

FIAP Pós-Tech — AI Scientist & Machine Learning Engineering | Tech Challenge (Fase 3)

Este documento formaliza a evolução do ecossistema preditivo de alfabetização infantil de um protótipo experimental isolado para uma infraestrutura de produção de nível corporativo (Enterprise). O foco das implementações descritas visa assegurar a escalabilidade, alta disponibilidade, monitoramento contínuo contra degradação estatística e governança estrita de qualidade de código (QA).

1. Arquitetura de Integração via API Assíncrona (FastAPI & Docker)

Para expor as predições de risco calculadas pelo pipeline do `scikit-learn` para sistemas externos unificados (como diários de classe digitais ou portais governamentais de Secretarias de Educação), desenvolveu-se uma camada de microserviço agnóstica.

1.1 Motor Assíncrono com FastAPI

- **Consistência e Validação Extrema:** Utilização da biblioteca `Pydantic` para definição e coerção automática de tipos no payload de entrada (contrato de dados estrito). Qualquer anomalia na estrutura impede o processamento imediato, evitando falhas ocultas.
- **Prevenção de Quebras de Contrato:** Alinhamento dinâmico automático na ordem e nome das variáveis com validação do atributo `feature_names_in_` do pipeline serializado, mitigando erros clássicos de dimensionalidade de matrizes em runtime.

1.2 Dockerização e Portabilidade Multipropósito

Para neutralizar o cenário clássico de incompatibilidade de ambientes de execução ("na minha máquina funciona"), o ecossistema foi integralmente containerizado via `Dockerfile` otimizado.

Estratégia de Arquitetura de Containers:

A imagem base utiliza `python:3.12-slim` para minimizar a superfície de ataque e o tamanho do artefato final. A execução em paralelo do servidor de visualização (Streamlit na porta `8501`) e do microserviço de predição (Uvicorn/FastAPI na porta `8000`) é orquestrada de forma otimizada através de forks paralelos nativos de sistema operacional, garantindo uma implantação enxuta de comando único para os avaliadores.

2. Monitoramento Avançado e Mitigação de Data Drift

Modelos de aprendizado de máquina sofrem inevitavelmente com a perda de aderência populacional ao longo do tempo. Mudanças em variáveis sociodemográficas ou em diretrizes de investimento alteram o padrão com que o classificador foi treinado.

2.1 Abordagem Estatística de Kolmogorov-Smirnov (KS-Test)

Para identificar antecipadamente a perda de performance analítica antes que ela impacte as decisões de políticas públicas, foi acoplada uma rotina autônoma de monitoramento de distribuição probabilística utilizando o teste bicaudal não paramétrico de Kolmogorov-Smirnov.

Métrica Executiva	Abordagem Técnica	Gatilho de Ação / Resolução
Hipótese Nula (H ₀)	As populações de produção seguem a mesma distribuição contínua do treino.	Modelo mantido estável em produção sem necessidade de alteração de pesos.
Hipótese Alternativa (H ₁)	A distribuição de pelo menos uma feature contínua divergiu significativamente.	Rejeição de H ₀ se p-value < 0.05. Disparo imediato de logs para retreinamento.

Esse motor atua diretamente sobre o fluxo de entrada, testando continuamente a aderência populacional das features cruciais identificadas pelo SHAP (como vulnerabilidade social e frequência escolar), impedindo falhas geradas por desvios macroeconômicos.

3. Governança de Software e Automação de CI/CD (GitHub Actions)

A maturidade da engenharia da solução é sustentada por uma esteira de Integração Contínua (CI) automatizada via GitHub Actions. O fluxo garante a qualidade de ponta a ponta sem intervenção manual de validação.

3.1 Esteira MLOps CI Flow

- Isolamento de Ambiente e Checkouts:** Em cada evento de `push` ou `pull_request` direcionado à branch `main` ou `feature/pipeline-ml`, uma máquina virtual linux limpa é instanciada pelo runner do GitHub.
- Instalação Automatizada e Caching:** O ambiente Python 3.12 é configurado de forma idêntica e as dependências do `requirements.txt` são montadas utilizando caches de pacotes para ganho de performance de esteira.
- Execução Rígida da Suíte de QA:** A ferramenta `pytest` realiza a varredura completa da pasta de testes unitários e de integração. Caso ocorra falha em testes de regressão de código ou de saída de predições, o `deploy` ou `merge` é automaticamente bloqueado na raiz.